

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY (HCMUT)
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



MACHINE LEARNING AND APPLICATIONS

INDIVIDUAL ASSIGNMENT

IMAGE CLASSIFICATION FOR FASHION-MNIST USING MLP

Instructor: Professor Nguyen An Khuong
Professor Le Hong Trang
Ph.D. Pham An Vinh

Student: Tran Thi Van Anh - 2570394

Ho Chi Minh City, 2026



Table of contents

1	Introduction	1
2	Exploring and Preprocessing data	1
2.1	Exploring data	1
2.2	Data preprocessing	2
2.2.1	Resize	2
2.2.2	Convert to Tensor	2
2.2.3	Normalization	2
3	Model architecture - Baseline	3
4	Loss function and optimizer choices	4
4.1	Loss function	4
4.2	Optimizer	4
5	Training Process and Strategies for Preventing Overfitting	5
5.1	Training configuration	5
5.2	Training process of the baseline model	5
5.3	Strategies of preventing overfitting	5
6	Hyperparameter experiments	6
6.1	Experiment 1: learning rate = 0.001	6
6.2	Experiment 2: learning rate = 0.003	6
6.3	Experiment 3: learning rate = 0.0001	7
7	Visualization	7
7.1	Baseline learning curves	7
7.2	Regularized Model Learning Curves	8
7.3	Fully Optimized Model Learning Curves	9
8	Evaluate performance	9
9	Advanced technique proposal	10
10	Conclusion	10



1 Introduction

Fashion-MNIST is a grayscale dataset, consisting of 10 categories, each represented by 6000 images in the training dataset and by 1000 in the test dataset. In this project, a Multi-Layer Perceptron (MLP) model was implemented to classify images from the FashionMNIST dataset.

The objective of this project is to predict the clothing category through preprocessing data, designing and training MLP architecture, loss function and optimizer choices, applying regularization techniques to reduce overfitting, experimenting with various hyperparameters, evaluating model and proposing advanced techniques to improve the model.

Source code: https://github.com/Vananhtran98/ML_Assignment_HK252_2570394

2 Exploring and Preprocessing data

2.1 Exploring data

The dataset is loaded in d2l book, which contains a totals of 70,000 grayscale image samples distributed across 10 different clothing object classes at 28x28 pixels resolution. It is labeled with numbers from 0 to 9, including:

Table 1: Fashion-MNIST Classes

Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

This study implements the PyTorch source code, using `torchvision.datasets.FashionMNIST` library to automatically load data (divided into 60,000 Train images and 10,000 Test images). To visualize according to experimental requirements, the system successfully extracted and randomly displayed a representative representing of 10 the object classes.

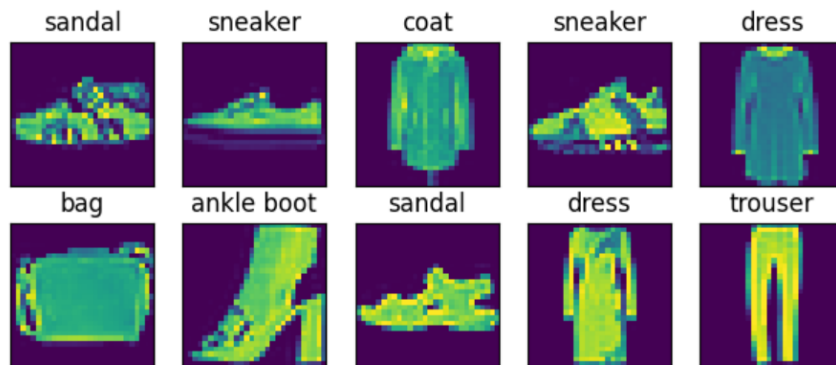


Figure 1: 10 sample images from the Fashion-MNIST dataset.

2.2 Data preprocessing

Data preprocessing plays a vital role in MLP. It helps to remove noise, normalize and bring features to a common scale. This process allows the network to converge faster, avoids vanishing/exploding gradient and improves the overall model performance. The preprocessing process is applied for the following techniques before training model.

2.2.1 Resize

The original image size was resized from 28×28 to 32×32 using:

```
transforms.Resize((32, 32))
```

This resizing ensures a consistent input size for the neural network.

2.2.2 Convert to Tensor

The original image pixel matrices were converted from the PIL Image format (with integer pixel values ranging from 0 to 255) into `torch.Tensor` format with floating-point values scaled to the range $[0.0, 1.0]$ using:

```
transforms.ToTensor()
```

This conversion allows the images to be processed efficiently by PyTorch.

2.2.3 Normalization

The image data was normalized using:

```
transforms.Normalize((0.5,), (0.5,))
```

This operation maps all pixel values into the symmetric range $[-1.0, 1.0]$, with a mean of 0 and a standard deviation of 1. Normalization helps reduce covariate shift, prevents excessive fluctuations in weight magnitudes, and allows the optimizer to converge toward a more stable minimum during training.

3 Model architecture - Baseline

The project builds a baseline architecture with an input layer, two fully-connected hidden layers and an output layer. It is designed to use sequential feed-forward structure with `nn.Sequential`, which is free of any advanced regularization techniques such as Dropout or Batch Normalization

[Input Image (32×32) $\rightarrow$ Flatten (1024) $\rightarrow$ Linear (256) $\rightarrow$ ReLU $\rightarrow$ Linear (128) $\rightarrow$ ReLU $\rightarrow$ Output (10)]

Model structure

```
FashionMLP(  
    (flatten): Flatten(start_dim=1, end_dim=-1)  
    (mlp): Sequential(  
        (0): Linear(in_features=1024, out_features=256, bias=True)  
        (1): ReLU()  
        (2): Linear(in_features=256, out_features=128, bias=True)  
        (3): ReLU()  
        (4): Linear(in_features=128, out_features=10, bias=True)  
    )  
)
```

Figure 2: Output of baseline MLP architecture.

Flatten layer

The input image with size $(32 \times 32 \times 1)$ is flattened into a one-dimensional feature vector containing $32 \times 32 \times 1 = 1024$ elements. This transformation allows the image data to be processed by the fully connected layers of the MLP model.

Hidden Layer 1 (1024 $\rightarrow$ 256)

The first hidden layer scales the 1024 input elements down to 256 neurons and is responsible for learning basic image features such as edges, shapes, and simple visual patterns.

Also, the `ReLU()` activation function is applied after this layer to introduce non-linearity and improve training efficiency.

Hidden Layer 2 (256 $\rightarrow$ 128)

The second hidden layer further refines the extracted features from the previous layer and helps the model distinguish between visually similar clothing categories.

The number of neurons is reduced from 256 to 128 in order to gradually compress the feature representation and reduce unnecessary information.

Output Layer (128 $\rightarrow$ 10)

The output layer contains 10 neurons corresponding to the 10 classes in the FashionMNIST dataset. Each neuron produces a logit value representing the prediction score for a specific class.

This simple baseline model serves as the empirical foundation for our project. Its primary purpose is to establish an initial performance benchmark, allowing to highlight severe overfitting bugs in next chapters and justify the necessity of advanced regularizations.

4 Loss function and optimizer choices

4.1 Loss function

CrossEntropyLoss is used because the task is a multi-class classification problem with 10 output classes.

```
criterion = nn.CrossEntropyLoss()
```

This loss function automatically applies the Softmax operation to convert the output logits into probability distributions and then calculates the classification error between the predicted labels and the true labels.

The Cross-Entropy Loss is calculated as:

$$L = - \sum_{j=1}^q y_j \log(\hat{y}_j) \quad (1)$$

where y_j represents the true label, $\hat{y}_j$ represents the predicted probability, and q is the total number of classes.

4.2 Optimizer

The Adam optimizer was selected for training:

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Adam is an extension of SGD that incorporates momentum and adaptive learning rates:

$$v_t = \beta_1 v_{t-1} + (1 - \beta_1) g_t \quad (2)$$

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2 \quad (3)$$

Here, β_1 and β_2 are weighting parameters, and g_t represents the gradient at time t .

Reasons for choosing Adam:

- Faster convergence.
- Stable training performance.
- Adaptive learning rate mechanism.
- Excellent performance for deep learning models.

5 Training Process and Strategies for Preventing Overfitting

5.1 Training configuration

The model is trained using mini-batch gradient descent. The specific hyperparameters are summarized in Table 2.

Table 2: Hyperparameters used for model training.

Parameter	Value
Batch Size	128
Epochs	40
Learning Rate	0.001
Patience	4

5.2 Training process of the baseline model

Table 3: Summary of model performance results.

Metric	Result
Train Accuracy	96.52%
Best Test Accuracy	89.46%
Final Test Loss	0.4666

The baseline model was initially trained without additional regularization techniques. During the early training epochs, both training accuracy and test accuracy improved steadily, indicating that the model successfully learned meaningful feature representations from the Fashion-MNIST dataset. However, after approximately Epoch 13, signs of overfitting began to appear. The training loss continued to decrease while the test loss gradually increased. At the same time, the training accuracy reached a high value of approximately 95%, whereas the improvement on the test dataset became limited.

This behavior indicates that the model started memorizing the training data instead of learning generalized patterns. As a result, the model's generalization capability decreased in later epochs.

5.3 Strategies of preventing overfitting

To improve the generalization performance and reduce overfitting, two regularization techniques were applied in this project, which are Dropout and Early Stopping.

Dropout

A Dropout layer with probability ($p = 0.2$) was inserted after the ReLU activation function in the hidden layer.

```
nn.Dropout(0.2)
```

During training, Dropout randomly disables 20% of neurons in each mini-batch. This mechanism prevents the network from depending too heavily on specific neurons and encourages the model to learn more robust feature representations.

Early Stopping

Early stopping was implemented using a patience value of 4 epochs.

patience = 4

The training process automatically stops when the test loss does not improve for four consecutive epochs. This strategy helps prevent excessive training and preserves the model at a better generalization state.

6 Hyperparameter experiments

To analyze the effect of hyperparameters on model performance, several experiments were conducted using different learning rates while keeping the remaining parameters unchanged.

The model was trained using the Adam optimizer with a batch size of 128 and Early Stopping regularization. Three learning rates were tested: (0.001), (0.003), and (0.0001).

6.1 Experiment 1: learning rate = 0.001

Table 4: Model performance with learning rate = 0.001

LR	Epoch stops	Train Acc	Test Acc	Train Loss	Test Loss
0.001	17	90.28%	88.94%	0.2637	0.3167

Analysis

After applying Dropout and Early Stopping with a learning rate of (0.001), the model achieved more stable training performance compared to the baseline model.

The best result was obtained around Epoch 13 with a test loss of (0.3149) and a test accuracy of (88.36%). Unlike the baseline model, the test loss remained relatively stable during later epochs, indicating reduced overfitting and improved generalization capability.

Since the test loss did not improve for four consecutive epochs, the Early Stopping mechanism automatically terminated the training process at Epoch 17. This helped prevent unnecessary training and preserved the model at a better generalization state.

6.2 Experiment 2: learning rate = 0.003

Table 5: Model performance with learning rate = 0.003

LR	Epoch stops	Train Acc	Test Acc	Train Loss	Test Loss
0.003	15	87.67%	87.69%	0.3373	0.3573

Analysis

When the learning rate was increased to (0.003), the model showed less stable training behavior compared to (0.001).

The test loss fluctuated more during training, indicating that the larger learning rate caused bigger weight updates and less smooth convergence. The best result was achieved at Epoch 11 with a test accuracy of (87.79%).

After that point, the test loss did not improve for four consecutive epochs, so the Early Stopping mechanism automatically stopped training at Epoch 15.

Compared to the (0.001) setting, the model converged faster but achieved slightly lower accuracy due to overshooting the optimal minimum.

6.3 Experiment 3: learning rate = 0.0001

Table 6: Model performance with learning rate = 0.0001

LR	Epoch stops	Train Acc	Test Acc	Train Loss	Test Loss
0.0001	40	91.29%	88.93%	0.2352	0.3079

Analysis

When using a smaller learning rate of (0.0001), the model showed a slower but more stable convergence process compared to the previous experiments.

The test loss decreased gradually and smoothly throughout training, indicating that the small learning rate produced more careful weight updates and reduced training instability. The best test loss of approximately (0.3046) was achieved around Epoch 38, while the highest test accuracy reached (89.26%).

Unlike the higher learning rate settings, the model continued making small improvements during later epochs. As a result, the Early Stopping mechanism was never activated because the test loss repeatedly achieved slightly better values before the patience counter reached its limit.

Therefore, the final model selects (0.001) as the optimal learning rate for this project.

7 Visualization

7.1 Baseline learning curves

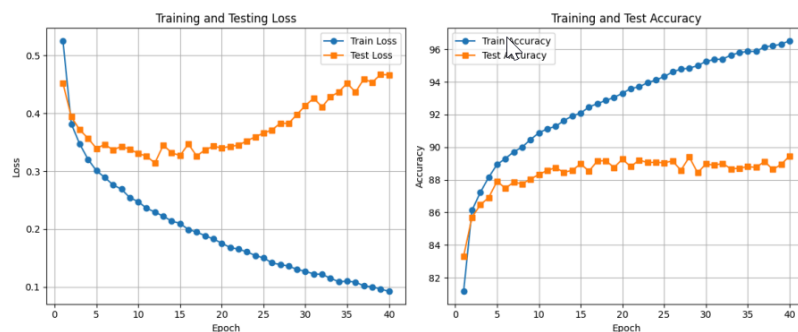


Figure 3: Without regularization curves

The learning curves clearly show signs of overfitting in the baseline model. During the early epochs, both training loss and test loss decreased steadily, indicating effective learning. However, after approximately Epoch 12, the test loss began to increase while the training loss continued to decrease.

Similarly, training accuracy continued improving throughout training, whereas test accuracy became stable and showed only minor improvements. The growing gap between training and test performance indicates reduced generalization capability.

In addition, a small fluctuation around Epoch 28 temporarily improved the test accuracy, which reset the Early Stopping counter and allowed the training process to continue longer than expected.

7.2 Regularized Model Learning Curves

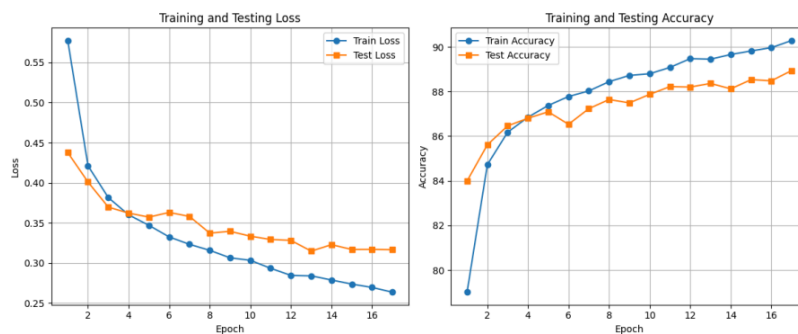


Figure 4: Regularized Learning Curves

The learning curves show that the model achieved stable training behavior after applying Dropout and Early Stopping with a learning rate of (0.001).

In the loss graph, both training loss and test loss decreased smoothly during the early epochs. The best test loss of approximately (0.3149) was achieved around Epoch 13. After that point, the test loss remained relatively stable instead of increasing sharply, indicating that the model successfully reduced overfitting.

The accuracy graph also demonstrates good generalization performance. The training accuracy and test accuracy stayed close to each other throughout training, resulting in a small generalization gap. At Epoch 17, the model achieved approximately (90.28%) training accuracy and (88.94%) test accuracy.

Since the test loss did not improve for four consecutive epochs after Epoch 13, the Early Stopping mechanism automatically terminated training at Epoch 17.

7.3 Fully Optimized Model Learning Curves

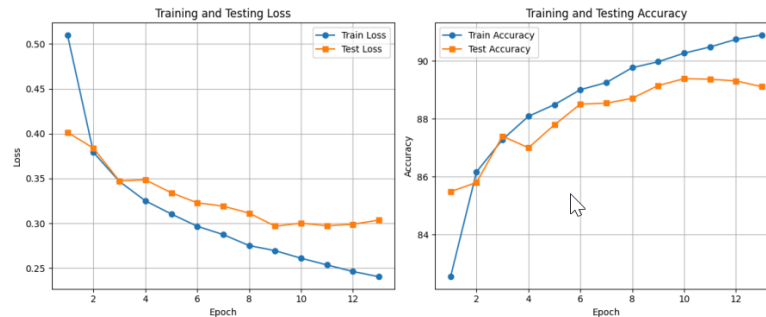


Figure 5: Fully Optimized Model Learning Curves

Figures demonstrate that the optimized model improves the stability of training and generalization performance of the MLP model.

In the loss graph, both training loss and test loss decreased smoothly during the early epochs. The model achieved its lowest test loss of approximately (0.2969) at Epoch 9, which was the best result among all experiments. After that point, the test loss remained stable without showing the sharp increase observed in the baseline model, indicating that overfitting was effectively controlled.

The accuracy graph also shows strong generalization capability. Training accuracy and test accuracy stayed close throughout training, resulting in a small generalization gap. The highest test accuracy of approximately (89.39%) was achieved around Epoch 10.

Since the test loss did not improve for four consecutive epochs after Epoch 9, the Early Stopping mechanism automatically terminated training at Epoch 13. Overall, the results show that this advanced structural optimization helped the model converge faster, achieve more stable learning behavior, and improve overall performance on the FashionMNIST dataset.

8 Evaluate performance

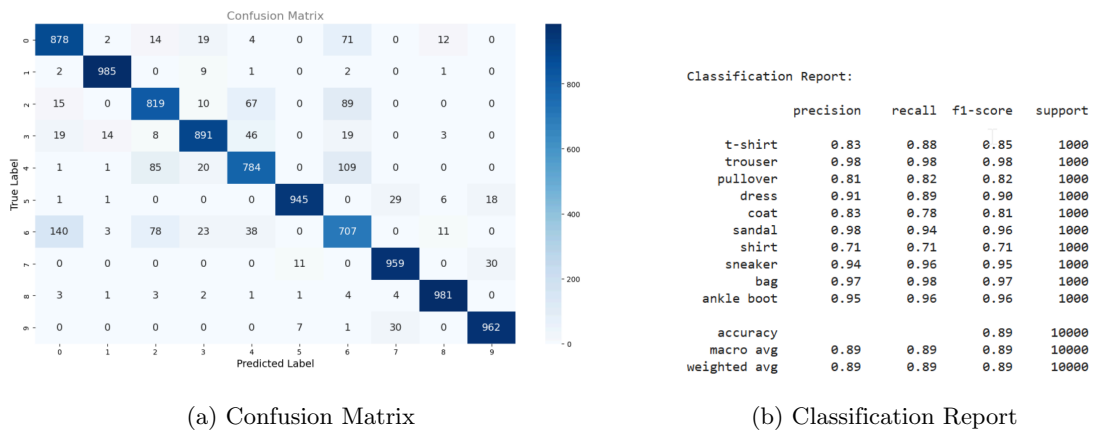


Figure 6: Model evaluation metrics

The confusion matrix and classification report demonstrate that the improved MLP model achieved strong classification performance on the FashionMNIST dataset, with an overall accuracy of approximately (89%).

Classes with distinctive visual characteristics such as Trouser, Bag, Sandal, and Ankle Boot achieved very high precision, recall, and F1-scores, indicating stable and accurate predictions. Most samples of these categories were correctly classified along the main diagonal of the confusion matrix.

However, the model still experienced confusion between visually similar upper-body clothing categories such as Shirt, Coat, and Pullover. Among them, the Shirt class obtained the lowest F1-score, showing that these categories remain challenging for a fully connected MLP architecture. In addition, the number of misclassifications between T-shirt and Shirt was reduced after applying Batch Normalization, indicating improved feature learning and training stability.

Overall, the results confirm that the fully optimized architecture, which integrates advanced optimization layers and Dropout, successfully improved the generalization capability of the model and reduced major classification errors on the test dataset.

9 Advanced technique proposal

The results in Chapter 7 and Chapter 8 show that Batch Normalization significantly improved the performance and stability of the MLP model. During training, the data distribution inside the hidden layers changes continuously, making optimization more difficult. This problem is known as Internal Covariate Shift.

To reduce this issue, Batch Normalization layers were added after the linear layers:

```
nn.BatchNorm1d(256)
nn.BatchNorm1d(128)
```

For each mini-batch, Batch Normalization normalizes the activations by adjusting their mean and variance. This helps keep the data distribution more stable during training.

As a result, the model converged faster, training became smoother, and the validation loss decreased more steadily. The improved model achieved a lower test loss of 0.2969 and better generalization performance without severe overfitting.

10 Conclusion

In this project, a Multi-Layer Perceptron model was successfully implemented for FashionMNIST image classification.

Several preprocessing techniques and regularization methods were applied to improve model performance. Hyperparameter tuning experiments showed that the learning rate significantly affects convergence and accuracy.

The final model achieved approximately 89% test accuracy. Regularization techniques such as Dropout and Early Stopping helped reduce overfitting and improve generalization.

For future work, more advanced architectures such as Convolutional Neural Networks (CNNs) could be explored to further improve classification performance.



Nguồn tham khảo

- [1] Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. Image classification dataset. Dive into Deep Learning, 2023. Accessed: 2026-05-22.